

Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	30 September 2026
Team ID	SWTID-2026-3024
Project Name	AI-Augmented Backend Development AI BlogNest API
Maximum Marks	4 marks

Technical Architecture

AI BlogNest follows a layered backend architecture designed to provide secure blog management, AI-powered content generation, database management, and RESTful API services.

Architecture Flow

Client / API User



REST API – Express.js Server



Authentication & Authorization Middleware



Controllers / Application Logic



Business Logic & AI Services



MongoDB Database / Google Gemini AI



JSON Response to Client

Major Architecture Components

- 1. Client / API Interface**
 - Users interact with the system through API requests.
 - Postman can be used to test and interact with the REST APIs.
 - The backend returns responses in JSON format.
- 2. Application Logic**
 - Node.js provides the runtime environment.
 - Express.js handles REST API routing and request processing.
 - Controllers manage authentication, blogs, and AI-related operations.
- 3. Authentication and Security**
 - JWT is used for authentication.
 - bcrypt.js is used for password hashing.
 - Role-based authorization controls access for Admin, Editor, Author, and Reader users.

4. **Database**
 - MongoDB is used as the primary database.
 - Mongoose provides database modeling and communication between the Node.js application and MongoDB.
 5. **AI Interface**
 - Google Gemini AI is integrated for AI-powered content generation and assistance.
 - The application sends appropriate prompts to the Gemini service and receives generated content.
 6. **Infrastructure**
 - The application can be developed and tested on a local system.
 - The backend runs on a Node.js server at port 5000 during development.
 - MongoDB can be accessed through MongoDB Atlas/cloud infrastructure.
 7. **External Interface**
 - Google Gemini API is used as the external AI service.
 - API requests and responses are handled through the backend application.
-

Table-1: Components & Technologies

S.No	Component	Description	Technology
1	User Interface	Users interact with the backend through REST API requests and API testing tools.	REST API, Postman, JSON
2	Application Logic-1	Handles API routing, request processing and server-side application logic.	Node.js, Express.js
3	Application Logic-2	Handles user authentication, password protection and role-based access control.	JWT, bcrypt.js, Express Middleware
4	Application Logic-3	Handles AI-powered blog content generation and AI assistance.	Google Gemini API
5	Database	Stores users, blogs, comments, categories, tags and related application data.	MongoDB, Mongoose
6	Cloud Database	Provides cloud-based database hosting and remote database access.	MongoDB Atlas
7	File Storage	Stores project configuration and application files required by the backend.	Local File System
8	External API-1	Provides AI-powered content generation and assistance.	Google Gemini API
9	External API-2	No separate external API is required for the core blog management functionality.	Not Applicable
10	Machine Learning Model	Generates AI-assisted blog content and provides intelligent content assistance.	Google Gemini Generative AI Model
11	Infrastructure (Server / Cloud)	Hosts and runs the backend application and database services.	Node.js Local Server, MongoDB Atlas

Table-2: Application Characteristics

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	Uses open-source technologies for backend development, API creation, database interaction and application development.	Node.js, Express.js, Mongoose, MongoDB
2	Security	Provides authentication, password hashing, role-based authorization, input validation, sanitization and protected API endpoints.	JWT, bcrypt.js, Express Middleware, Validation & Sanitization
3	Scalable Architecture	The layered architecture separates API routing, authentication, controllers, business logic, database access and AI services. This allows individual components to be enhanced or scaled independently.	Node.js, Express.js, MongoDB, Modular Architecture
4	Availability	The backend can be hosted on a continuously running server, while MongoDB Atlas provides cloud-based database availability.	Node.js Server, MongoDB Atlas
5	Performance	REST APIs provide lightweight JSON communication. Database indexing/search mechanisms, efficient queries, validation and modular processing help improve application performance.	Express.js, MongoDB, Mongoose, JSON REST API

Technology Stack Summary

Frontend / Client

- Postman / REST API Client
- JSON-based API communication

Backend

- Node.js
- Express.js

Database

- MongoDB
- Mongoose ODM
- MongoDB Atlas

Authentication & Security

- JSON Web Token (JWT)

- bcrypt.js
- Role-Based Access Control
- Input Validation
- Data Sanitization
- Rate Limiting

Artificial Intelligence

- Google Gemini AI
- Generative AI Model
- AI Blog Content Generation
- AI Content Assistance
- AI Summarization

Development Environment

- Visual Studio Code
- Node.js
- npm
- Localhost Server

Architecture Characteristics

1. Modular Architecture

AI BlogNest follows a modular backend structure where authentication, blog management, AI functionality, database operations and other services are organized into separate modules.

2. RESTful Architecture

The application exposes REST API endpoints through Express.js. HTTP methods such as GET, POST, PUT/PATCH and DELETE are used according to the operation being performed.

3. Database-Driven Architecture

MongoDB stores application data, while Mongoose provides schemas, models and database interaction from the Node.js application.

4. AI-Integrated Architecture

Google Gemini AI is connected to the backend through an external API interface. The backend processes user requests, sends appropriate prompts to the AI service and returns the generated result.

5. Secure Architecture

Authentication and authorization mechanisms protect application resources. Passwords are hashed using bcrypt.js, while JWT tokens are used to authenticate users and control access to protected endpoints.

6. Cloud-Ready Architecture

Although the application can be developed and tested locally, MongoDB Atlas provides cloud database support, making the system suitable for future cloud deployment and scaling.

Local and Cloud Demarcation

Local Environment

- Node.js runtime
- Express.js backend server
- Visual Studio Code
- Postman for API testing
- Local project files
- Local development server

Cloud Environment

- MongoDB Atlas for cloud database hosting
 - Google Gemini API for AI services
-

External Interfaces

The primary external interface used by AI BlogNest is the **Google Gemini API**.

The backend communicates with the Gemini service to provide AI-powered functionality such as:

- AI blog content generation
- AI content assistance
- AI summarization
- Intelligent content recommendations where applicable

The external AI response is processed by the backend before being returned to the API client.

Data Storage

MongoDB is used as the primary data storage system.

The database can store information related to:

- Users
- Authentication details
- Roles
- Blog posts
- Categories
- Tags
- Comments
- Media metadata
- Other application-related records

Mongoose is used as the ODM layer to define schemas and interact with MongoDB.

Machine Learning / AI Model Interface

AI BlogNest integrates Google Gemini through an API-based interface.

Flow:

User Request



Express.js REST API



AI Controller / Service



Google Gemini API



Generated AI Content



Backend Processing



JSON Response

This integration enables AI BlogNest to reduce manual content creation effort and provide intelligent assistance to users.